Java Beans – Content + Simple Programs

1. Introduction to JavaBeans

A **JavaBean** is a reusable software component written in Java.
It follows three main rules:

1. Must have a **public no-arg constructor**
2. Must have **private properties** with **public getters and setters**
3. Must be **serializable** (i.e., implement `Serializable`)

✔ Simple Example – Basic JavaBean

```
import java.io.Serializable;

public class StudentBean implements Serializable {
    private String name;
    private int age;

    public StudentBean() {}  // no-arg constructor

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

---

2. Advantages of JavaBeans

- Reusable components
- Easy to maintain
- Platform-independent
- Follows standard naming conventions
- Supports tools like IDE builders (NetBeans, Eclipse)

(No code required — conceptual topic.)

---

3. Introspection

Introspection = discovering a bean's properties, methods, events at runtime.
Java provides:

- `Introspector`
- `BeanInfo`
- `PropertyDescriptor`

✔ Simple Introspection Program

```
import java.beans.*;

public class BeanIntrospectionDemo {
    public static void main(String[] args) throws Exception {
        BeanInfo info = Introspector.getBeanInfo(StudentBean.class);

        System.out.println("Properties:");
        for (PropertyDescriptor pd : info.getPropertyDescriptors()) {
            System.out.println(" - " + pd.getName());
        }

        System.out.println("\nMethods:");
        for (MethodDescriptor md : info.getMethodDescriptors()) {
            System.out.println(" - " + md.getName());
        }
    }
}
```

---

4. Bound and Constrained Properties

Bound Property

Notifies listeners whenever a property value changes using `PropertyChangeSupport`.

✔ Bound Property Example

```
import java.beans.*;

public class CounterBean {
    private int count;
    private PropertyChangeSupport pcs = new PropertyChangeSupport(this);

    public void addPropertyChangeListener(PropertyChangeListener listener) {
        pcs.addPropertyChangeListener(listener);
    }

    public int getCount() { return count; }

    public void setCount(int newCount) {
        int oldCount = this.count;
        this.count = newCount;
```

```
        pcs.firePropertyChange("count", oldCount, newCount);
    }
}
```

✔ Using the Bean

```
public class TestBound {
    public static void main(String[] args) {
        CounterBean bean = new CounterBean();
        bean.addPropertyChangeListener(evt ->
            System.out.println("Count changed from " + evt.getOldValue() + "
to " + evt.getNewValue())
        );

        bean.setCount(1);
        bean.setCount(5);
    }
}
```

Constrained Property

A property change can be **vetoed** using `VetoableChangeSupport`.

✔ Constrained Property Example

```
import java.beans.*;

public class AgeBean {
    private int age;
    private VetoableChangeSupport vcs = new VetoableChangeSupport(this);

    public void addVetoableChangeListener(VetoableChangeListener listener) {
        vcs.addVetoableChangeListener(listener);
    }

    public int getAge() { return age; }

    public void setAge(int newAge) throws PropertyVetoException {
        int oldAge = this.age;
        vcs.fireVetoableChange("age", oldAge, newAge);
        this.age = newAge;
    }
}
```

✔ Using the Constrained Bean

```
public class TestConstrained {
    public static void main(String[] args) {
        AgeBean bean = new AgeBean();

        bean.addVetoableChangeListener(evt -> {
```

```
            if ((int) evt.getNewValue() < 0)
                throw new PropertyVetoException("Age cannot be negative!",
evt);
        });

        try {
            bean.setAge(20);   // OK
            bean.setAge(-5);   // VETO
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

---

5. Persistence

Beans are often **saved and restored** using serialization.

✔ Simple Persistence Example

Save the bean

```
import java.io.*;

public class SaveBean {
    public static void main(String[] args) throws Exception {
        StudentBean s = new StudentBean();
        s.setName("John");
        s.setAge(22);

        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("student.ser"));
        out.writeObject(s);
        out.close();

        System.out.println("Bean saved.");
    }
}
```

Load the bean

```
import java.io.*;

public class LoadBean {
    public static void main(String[] args) throws Exception {
        ObjectInputStream in = new ObjectInputStream(new
FileInputStream("student.ser"));
        StudentBean s = (StudentBean) in.readObject();
        in.close();

        System.out.println("Loaded: " + s.getName() + ", " + s.getAge());
    }
```

```
}
```

---

6. Customizers

Customizers give GUI tools (like NetBeans) a way to visually configure beans.

✔ Simple Customizer Stub

(Customizers usually require UI, so this is minimal.)

```
import java.awt.*;
import java.beans.*;

public class StudentCustomizer extends Panel implements Customizer {
    private StudentBean bean;
    private TextField nameField = new TextField(10);

    public StudentCustomizer() {
        setLayout(new FlowLayout());
        add(new Label("Name:"));
        add(nameField);

        nameField.addTextListener(e -> bean.setName(nameField.getText()));
    }

    @Override
    public void setObject(Object bean) {
        this.bean = (StudentBean) bean;
    }
}
```

---

7. JavaBeans API

Java provides several classes in the **java.beans** package:

- `Introspector`
- `PropertyDescriptor`
- `BeanInfo`
- `PropertyChangeSupport`
- `VetoableChangeSupport`
- And more…

✔ Simple API Demonstration

```
import java.beans.*;

public class BeanAPIDemo {
```

```
    public static void main(String[] args) throws Exception {

        BeanInfo info = Introspector.getBeanInfo(StudentBean.class);

        System.out.println("Using JavaBeans API:");
        for (PropertyDescriptor pd : info.getPropertyDescriptors()) {
            System.out.println("Property: " + pd.getName());
        }
    }
}
```

✔ All main topics covered with simple programs